

Deep Learning Project – Methodology Analysis on MNIST

Student Names:

Rayyan Salameh

Yazan Qasem

Abdulrahman Abukarsh

Mohammad Rahmani

1. Introduction

In this project, we applied the main deep learning concepts that we studied in the course laboratory sessions on the MNIST dataset.

The goal was not just to build a model, but to understand how different methodologies affect performance and generalization.

We focused on a few important methods and tested them in depth instead of trying to use everything superficially.

2. Dataset

We used the MNIST dataset, which contains handwritten digits from 0 to 9.

- 70,000 images in total
- 60,000 for training
- 10,000 for testing
- Image size: 28×28 (grayscale)

Dataset Quality

The dataset is:

- Balanced across all classes
- Clean with no missing data
- Easy to work with

****Note:****

MNIST is relatively simple, so it is good for testing ideas, but it does not fully represent real-world complexity.

3. Methods Used

We implemented the following methods from the course:

- Baseline model (linear)
- Deep Neural Network (MLP)
- Activation functions:
 - ReLU
 - Sigmoid
 - Leaky ReLU
- Loss functions:
 - CrossEntropy
 - MSE
- Regularization:
 - L1
 - L2 (weight decay)
 - Dropout
 - Batch Normalization
 - Label Smoothing
 - Early Stopping
- Optimization:
 - SGD with momentum
 - Minibatches
- FGSM (adversarial test)

4. Model Design

Baseline vs Deep Model

We started with a simple linear model (baseline), then moved to a deeper model with hidden layers.

****Why?****

Because the baseline model can only learn simple patterns, while deep models can capture more complex relationships.

Activation Functions

We tested:

- ReLU
- Sigmoid
- Leaky ReLU

****Why?****

- ReLU is fast and avoids vanishing gradients
- Sigmoid was used to show its limitations
- Leaky ReLU helps prevent dead neurons

Loss Functions

We compared:

- CrossEntropy
- MSE

****Why?****

CrossEntropy is better for classification because it gives stronger gradients.
MSE was used just to show that it is not suitable for this type of problem.

Regularization

In the main model, we used:

- L1 and L2
- Dropout
- BatchNorm
- Label Smoothing
- Early Stopping

****Why?****

These methods help reduce overfitting and improve generalization.

Optimization

We used:

- SGD with momentum (0.9)
- Learning rate = 0.01

- Batch size = 64

****Why?****

Momentum helps speed up training and makes it more stable.

5. Hyperparameter Tuning

We tried different values before choosing the final ones.

- Learning rate:

- 0.1 → unstable
- 0.001 → too slow
- 0.01 → best balance

- Batch size:

- 32 → better accuracy but slower
- 128 → faster but slightly worse
- 64 → best choice

- Dropout:

- 0.2 vs 0.5
- 0.5 worked better

We selected the final values based on validation performance.

6. Experiments

We ran the following experiments:

1. Baseline model
2. Deep model (ReLU)
3. Deep model (Sigmoid)
4. Deep model (Leaky ReLU)
5. Deep model with MSE
6. Deep model without regularization

##7. RESULTS

The baseline model achieved a test accuracy of 90.80%.

The deep model using ReLU achieved the highest performance with 97.92%.

The Sigmoid-based model performed worse, reaching 91.30%.

The Leaky ReLU model performed very similarly to ReLU with 97.85%.

The model trained with MSE loss achieved 95.90%, which is lower than CrossEntropy-based models.

Finally, the model without regularization achieved 97.19%, showing slightly worse generalization compared to the fully regularized model.

Analysis

- Deep models are much better than the baseline
- ReLU gave the best results
- Sigmoid performed poorly due to vanishing gradient
- MSE is worse than CrossEntropy for classification
- Regularization improved generalization

8. Combined Methods

The best model used all of these together:

- ReLU
- CrossEntropy
- SGD + momentum
- L1 + L2
- Dropout
- BatchNorm
- Label smoothing
- Early stopping

This combination gave the highest accuracy.

9. Robustness (FGSM)

We applied FGSM with $\epsilon = 0.1$.

The accuracy dropped under attack, which shows that the model is sensitive to small input changes.

10. Implementation Details

- Framework: PyTorch
- Epochs: 5–20
- Hardware: CPU / GPU
- Optimizer: SGD

11. Conclusion

This project showed that choosing the right methods is very important.

The best performance came from:

- Deep model
- ReLU activation
- CrossEntropy loss
- Regularization techniques

12. Final Insight

Good results do not come from complexity alone, but from using the right methods in the right way.